Water Management Network

Práctica no guiada: Sockets, Streaming de Eventos,

Colas y modularidad.

SISTEMAS DISTRIBUIDOS



Universitat d'Alacant
Universidad de Alicante

David Blasco Pérez - 78133139T
Aitana López Lloret - 48800385C
Curso 2026/2027

1. Introducción y Arquitectura del Sistema **2**
1.1. Objetivo del proyecto 2
1.2. Topología de red 3
1.3. Componentes 3
2. Decisiones Técnicas: Concurrencia, Resiliencia y Red **4**
2.1. Gestión de Concurrencia y Prevención de Starvation en el Monitor: 4
2.2. Tolerancia a Fallos y Máquina de Estados (Resiliencia): 4
2.3. Hilos y Colas en el Engine (Interfaz Local): 4
2.4. Consumidores Dinámicos en Kafka (WM_FO): 5
2.5. Despliegue y Orquestación con Docker: 5
3. Decisiones Técnicas: Lógica Central, Base de Datos y Protocolo **5**
3.1. Diseño del Protocolo de Capa de Enlace 5
3.2. Arquitectura de WM_Central: Gestión multihilo 5
3.3. Integración con Base de Datos y Resiliencia 5
3.4. Integración de Productores Kafka y Enrutamiento 6
4. Usabilidad y Entorno de Pruebas **6**
4.1. Códigos ANSI y Logging sin bloqueos 6
4.2. Ejecución Autónoma de Servicios (Modo Batch) 6
5. Conclusiones y Posibles Mejoras **7**
Conclusión: 7
Mejoras futuras: 7
6. Guía de despliegue **7**
6.1. Despliegue del Entorno Cloud (Railway / AWS) 7
6.2. Despliegue en Laboratorio 2: Estaciones de Riego (WM_WS) 7
6.3. Despliegue en Laboratorio 1: Operarios (WM_FO) 8
7. Repositorio, Código Fuente y Uso de Inteligencia Artificial **8**
7.1. Estructura del Repositorio 8
7.2. Trazabilidad y Uso de IA 8
8 . Anexos: Evidencias de funcionamiento **9**

1. Introducción y Arquitectura del Sistema

1.1. Objetivo del proyecto

El presente proyecto tiene como objetivo el diseño, desarrollo y despliegue de un **Sistema Distribuido de Gestión de Riegos**. Esta plataforma permite la monitorización, control y telemetría en tiempo real de múltiples estaciones de riego agrícola de forma remota. El sistema ha sido diseñado bajo estrictos requisitos de alta disponibilidad, tolerancia a fallos y concurrencia, garantizando que el estado físico de las electroválvulas y las métricas de volumen de agua sean precisas y consistentes, incluso ante interrupciones de red o caídas temporales de los nodos centrales.

1.2. Topología de red

Para resolver las necesidades de comunicación, la arquitectura implementa un modelo híbrido que combina comunicaciones síncronas y asíncronas, desplegado íntegramente sobre contenedores **Docker** utilizando una red virtual de tipo *bridge* (red-ws01).

- **Comunicación Cliente-Servidor (Sockets TCP):** Constituye la red operativa crítica del sistema. Enlaza los componentes físicos y lógicos (Engine, Monitor y Central) mediante conexiones TCP. Para garantizar la integridad de los datos en la capa de aplicación, se ha diseñado un protocolo propio basado en tramas delimitadas por caracteres de inicio y fin (<STX> y <ETX>), junto con un mecanismo de validación por redundancia longitudinal (<LRC>).
- **Comunicación Publicador-Suscriptor (Apache Kafka):** Se emplea para desacoplar a los usuarios humanos (operarios de campo) de la red operativa. A través de un *broker* de mensajería asíncrona, los operarios publican peticiones de riego y se suscriben a tópicos de respuesta, evitando que la Central se bloquee gestionando interfaces de usuario y garantizando el enrutamiento escalable de los eventos.



1.3. Componentes

El ecosistema distribuido se divide en cuatro módulos principales, cada uno con responsabilidades claramente delimitadas:

- **WM_Central (Cloud):** Ejerce de nodo coordinador o servidor principal. Mantiene un registro concurrente de todas las estaciones de riego activas, procesa las peticiones asíncronas provenientes de Kafka, orquesta las órdenes de inicio y fin de riego hacia las estaciones correspondientes, y consolida las métricas finales en la base de datos persistente.
- **WM_WS_M (Monitor):** Actúa como pasarela (Gateway) y supervisor de la estación física (Edge Computing). Mantiene una conexión persistente con la nube y actúa como servidor

local para el Engine. Su función es aislar los fallos: evalúa periódicamente la "salud" del Engine local, detecta averías físicas (fugas), y garantiza mediante un sistema de colas temporales que no se pierdan datos si la Central sufre una desconexión.

- **WM_WS_E (Engine):** Simula la placa de control de hardware de la estación. Físicamente, gestiona la apertura y cierre de la válvula y simula las lecturas de caudal y volumen acumulado. Dispone de un hilo de ejecución independiente que habilita un menú interactivo local, permitiendo intervenciones manuales de urgencia directas sobre el terreno.
- **WM_FO (Operario / Field Operator):** Es la interfaz de usuario en campo. Actúa exclusivamente como cliente de Kafka (Productor y Consumidor). Permite a los operarios solicitar riegos manuales de forma interactiva o ejecutar despliegues masivos simulados mediante la lectura automatizada de ficheros de configuración (servicios.txt).

2. Decisiones Técnicas: Concurrencia, Resiliencia y Red

2.1. Gestión de Concurrencia y Prevención de *Starvation* en el Monitor:

Para que el Monitor (WM_WS_M) atendiera simultáneamente a la Central y al Engine sin sufrir bloqueos ni retrasos por *timeouts* prolongados (los cuales provocaban desincronización y detección de falsas averías), se implementó un bucle de eventos no bloqueante. Al configurar los *timeouts* de ambos *sockets* a 0.05 segundos, el Monitor intercala su atención en tiempo real, vaciando los búferes de lectura instantáneamente y garantizando que las métricas de riego lleguen a la Central exactamente cada segundo.

2.2. Tolerancia a Fallos y Máquina de Estados (Resiliencia):

Aislamiento de hardware: Se definió por diseño que, si el Monitor está activo pero pierde conexión con el Engine local (o este notifica un error), la estación asume inmediatamente el estado crítico de FUGA (avería). Esto impide que la Central asigne peticiones a una válvula físicamente inoperativa.

Autonomía ante caída de la nube: Se integró un patrón *Store & Forward* en el Monitor para mitigar desconexiones de WM_Central. Si la red principal cae durante un servicio, el Engine sigue regando de forma autónoma. El Monitor almacena las métricas temporalmente y, al recuperarse la conexión, purga los datos redundantes transmitiendo únicamente el volumen total acumulado para evitar la saturación de la red.

2.3. Hilos y Colas en el Engine (Interfaz Local):

Para cumplir con el requisito de accionamiento manual en la estación sin interrumpir el envío continuo de la telemetría, se extrajo la lectura del teclado (input()) a un hilo independiente (*daemon*). La comunicación entre este hilo interactivo y el hilo principal de red se realiza mediante una cola segura (queue.Queue), garantizando la seguridad en la concurrencia (*thread-safety*) y evitando colisiones de paquetes al enviar comandos hacia la Central.

2.4. Consumidores Dinámicos en Kafka (WM_FO):

Para solucionar la lectura accidental de respuestas obsoletas de ejecuciones pasadas (causada por el uso de un group_id estático en el *broker*), se modificó la suscripción del operario añadiendo una marca de tiempo dinámica al identificador (f"fo-{ID}-{int(time.time())}") junto a la directiva auto_offset_reset='latest'. Esto asegura que cada ejecución del FO actúe como un cliente completamente nuevo, procesando de forma exclusiva los mensajes en tiempo real.

2.5. Despliegue y Orquestación con Docker:

Se descartó el uso de redes *host* estándar en favor de una red *bridge* personalizada (red-ws01), lo que aísla el tráfico de las estaciones y habilita la resolución de nombres DNS nativa de Docker. El levantamiento del escenario se automatizó mediante un *script* de Bash (lanzar_estaciones.sh), el cual permite instanciar docenas de parejas Monitor-Engine en modo *detached* (-dit) en segundos, inyectando los parámetros de red mediante variables de entorno para evitar *hardcodear* direcciones IP en el código fuente.

3. Decisiones Técnicas: Lógica Central, Base de Datos y Protocolo

3.1. Diseño del Protocolo de Capa de Enlace

El protocolo de comunicación local y remoto se ha centralizado en el módulo common.protocol. Para garantizar la integridad de las órdenes industriales (apertura/cierre de válvulas), las tramas de datos se envuelven utilizando bytes de control estándar: <STX> (inicio) y <ETX> (fin). A esto se le suma un byte <LRC>, calculado mediante una operación lógica XOR sobre todos los bytes de datos. Cuando la Central (o el Monitor) recibe una trama, recalcula este LRC, si el valor no coincide con el enviado o el formato está corrompido, se descarta inmediatamente el mensaje y se responde con un byte <NACK>. Esto fuerza al emisor a retransmitir el paquete hasta 3 veces antes de dar la conexión por perdida. Si la trama es perfecta, se asiente con <ACK>.

3.2. Arquitectura de WM_Central: Gestión multihilo

La Central (WM_Central.py) actúa como el cerebro del sistema utilizando una arquitectura TCP multihilo. Por cada Monitor que se conecta, el bucle principal (servidor.accept()) delega el control en un subhilo independiente (manejar_hilo). La concurrencia entre los comandos enviados por los Operarios y los Sockets de red se gestiona mediante buzones individuales asíncronos (queue.Queue) asignados a cada estación.

3.3. Integración con Base de Datos y Resiliencia

La persistencia de estados y métricas se ha resuelto utilizando SQLite3 a través del módulo db_manager.py. Para evitar condiciones de carrera y bloqueos habituales en entornos multihilo, cada función abre y cierra su propia conexión atómica a la base de datos. La lógica de negocio implementa una máquina de estados resiliente:

- Si la Central se reinicia abruptamente, cierra de forma segura todos los riegos "huérfanos" asignándoles el motivo REINICIO_CENTRAL.

- Si una estación que estaba regando se reconecta y comienza a emitir métricas de nuevo, la Central recupera automáticamente su sesión abierta y reasigna a su Operario (FO) sin pérdida de datos.

- Se incorpora una función correctora (corregir_volumen_ultimo_riego) para actualizar el histórico si una estación caída logra enviar tardíamente su volumen real de agua consumida.

3.4. Integración de Productores Kafka y Enrutamiento

La Central integra Apache Kafka para separar el tráfico de control físico (Sockets) de las interfaces de usuario (Operarios). Un hilo demonio (iniciar_kafka) consume continuamente el topic peticiones_operarios. Al recibir una orden de riego, la Central valida la existencia del operario y la disponibilidad de la estación consultando la base de datos. Si es válida, mete la orden en el buzón (Queue) de la estación. Posteriormente, la Central utiliza un productor de Kafka para enrutar las respuestas hacia los operarios en el topic respuesta_central, categorizándolas como confirmaciones de progreso (INFO), rechazos/averías (ERROR) o mensajes de difusión global ineludibles (ALERTA#ALL) en caso de fugas.

4. Usabilidad y Entorno de Pruebas

4.1. Códigos ANSI y Logging sin bloqueos

Al carecer de interfaz gráfica tradicional, se ha diseñado un entorno de consola. Se emplean códigos de escape ANSI para colorear la salida del terminal, destacando en verde las conexiones exitosas y en rojo los errores críticos o averías simuladas. Los logs incluyen etiquetas descriptivas ([INFO], [ALERTA], [MÉTRICAS]) que permiten una trazabilidad visual instantánea. Para visualizar el estado global sin "spam" de red, se implementó un panel visual dinámico (panel.py). Este panel intercepta temporalmente las impresiones de los demás hilos (usando un búfer de io.StringIO()) para que la tabla de colores y las métricas en tiempo real no se descuadren, volcando los registros acumulados ordenadamente solo cuando el usuario sale del panel.

4.2. Ejecución Autónoma de Servicios (Modo Batch)

Para garantizar la fiabilidad del sistema bajo estrés, el cliente de los operarios (WM_FO.py) incluye una funcionalidad de ejecución por lotes. Un hilo en segundo plano (ejecutar_fichero) lee comandos desde un archivo de texto (servicios.txt), permitiendo programar baterías masivas de riegos secuenciales. El sistema procesa cada línea: envía la solicitud de riego, aguarda dinámicamente hasta que la Central confirma el inicio, cronometra la duración estipulada (DURACION_RIEGO), envía la petición de cierre y aplica una pausa reglamentaria (ESPERA_ENTRE_SERVICIOS de 4 segundos) antes de conmutar a la siguiente estación. Esto permite testear cuellos de botella y comprobar la robustez de las validaciones concurrentes de la Central.

5. Conclusiones y Posibles Mejoras

Conclusión:

El proyecto ha cumplido con los requisitos establecidos en la especificación, logrando la implementación funcional de un Sistema Distribuido altamente resiliente. La segmentación en dos redes operativas paralelas (comunicación síncrona TCP para el hardware de campo y asíncrona vía Kafka para las interfaces de usuario) ha demostrado ser un diseño excelente para evitar cuellos de botella y aislar errores. La máquina de estados implementada en la nube soporta caídas abruptas de cualquier nodo, interrupciones de red y averías de hardware físico (fugas), manteniendo la integridad del volumen hídrico contabilizado y recuperando las sesiones huérfanas con éxito sin comprometer la base de datos.

Mejoras futuras:

- **Capa de Seguridad (TLS):** Incorporar envoltura SSL/TLS a los sockets TCP entre el Monitor y la Central. En un escenario de producción real, las tramas de control de electroválvulas no deben viajar en texto plano por redes públicas.
- **Interfaz de Usuario (Frontend):** Sustituir o complementar las aplicaciones de consola interactiva (WM_FO y el menú de WM_Central) por una API RESTful acoplada a un *dashboard* web (por ejemplo, React) y una aplicación móvil para los operarios de campo, mejorando drásticamente la experiencia de usuario.

6. Guía de despliegue

Para garantizar la reproducibilidad del entorno durante la evaluación, el sistema se divide en una infraestructura Cloud (backend) y nodos locales (Edge). A continuación se detallan los pasos exactos de despliegue.

6.1. Despliegue del Entorno Cloud (Railway / AWS)

La infraestructura central debe estar completamente operativa antes de levantar los laboratorios físicos.

1. **Broker de Kafka y Base de Datos:** Los servicios administrados de Apache Kafka y SQLite3 se encuentran aprovisionados y expuestos en la nube.
2. **WM_Central:**
    - Construir la imagen de la Central: docker build -t wm_central ./WM_Central
    - Ejecutar el contenedor exponiendo el puerto TCP: docker run -d -p <PUERTO_ESCUCHA>:<PUERTO_ESCUCHA> --name central_node wm_central <PUERTO_ESCUCHA> <IP_KAFKA>

6.2. Despliegue en Laboratorio 2: Estaciones de Riego (WM_WS)

En el primer equipo físico del laboratorio se instanciarán los simuladores de hardware.

1. Navegar al directorio de las estaciones: cd WM_WS
2. Levantar múltiples estaciones de forma automatizada mediante el script de bash diseñado:./lanzar_estaciones.sh <IP_CENTRAL> <PUERTO_CENTRAL> <IP_KAFKA>*(Este script construye las imágenes de Monitor y Engine, crea la red bridge interna red-ws01, e instancia pares de contenedores acoplados por cada estación, pasándoles las variables de entorno necesarias).*
3. Para intervenir manualmente una estación y simular una avería de hardware: docker attach <nombre_contenedor_engine>

6.3. Despliegue en Laboratorio 1: Operarios (WM_FO)

En el segundo equipo físico se instanciarán las interfaces de usuario.

1. Navegar al directorio de operarios: cd WM_FO
2. Construir la imagen: docker build -t wm_fo .
3. Lanzar consolas interactivas para cada operario:docker run -it --rm -e KAFKA_SERVER=<IP_KAFKA> -e ID_OPERARIO=<ID> wm_fo
4. Para la ejecución en modo "Batch" automatizado, montar el volumen del archivo de texto:docker run -it --rm -v $(pwd)/servicios.txt:/app/servicios.txt -e KAFKA_SERVER=<IP_KAFKA> wm_fo

7. Repositorio, Código Fuente y Uso de Inteligencia Artificial

7.1. Estructura del Repositorio

El código fuente se encuentra alojado en GitHub cumpliendo con la estructura requerida (carpetas WM_Central, WM_WS, WM_FO y ficheros Readme.md / Nombres.txt). El profesorado cuenta con permisos de acceso directo para la revisión del control de versiones.

7.2. Trazabilidad y Uso de IA

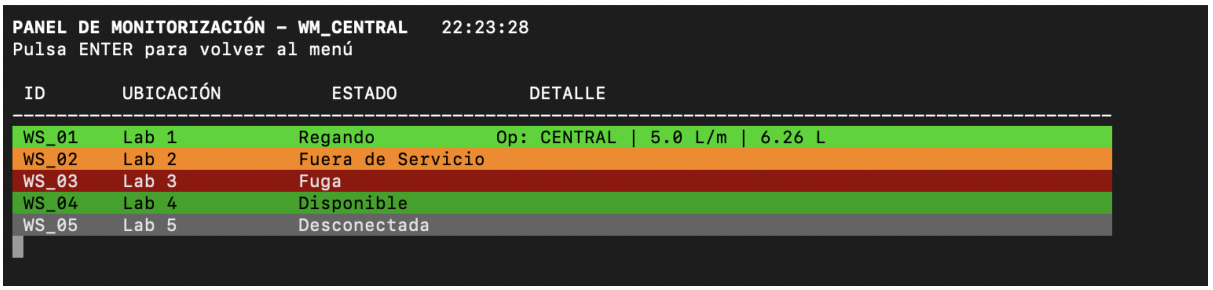
Conforme a las directrices de la práctica, se ha utilizado Inteligencia Artificial exclusivamente como herramienta de apoyo arquitectónico y resolución de bloqueos técnicos, manteniendo en todo momento la autoría humana sobre la lógica del código. Los prompts utilizados han sido documentados en el directorio /prompts del repositorio. Entre las consultas más relevantes para la superación de retos técnicos destacan:

- *Arquitectura de Red:* "¿Cómo diseño un bucle asíncrono con BlockingIOError para que el Monitor actúe como puente bidireccional entre la Central y el Engine?"
- *Tolerancia a fallos:* "¿Cómo implemento un periodo de gracia en Python para retrasar la evaluación de un socket y evitar falsas alarmas de avería?" y "¿Cómo capturo la señal SIGTERM en Docker para un cierre elegante del socket?"

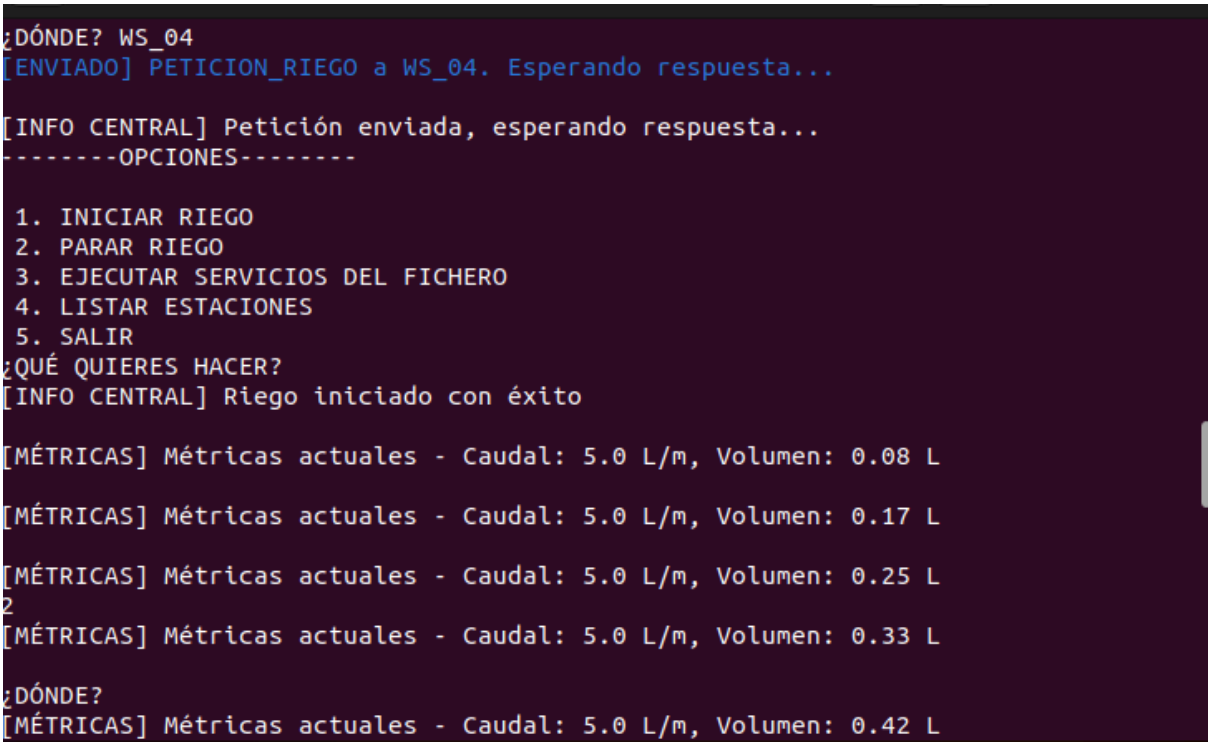
- *Gestión de concurrencia:* "¿Cómo utilizo threading para ejecutar un consumidor de Kafka en segundo plano sin bloquear el menú interactivo principal?"
- *Orquestación:* "¿Cómo creo un script en Bash que construya imágenes Docker, elimine contenedores previos y lance múltiples terminales interactivas automáticamente?"

8 . Anexos: Evidencias de funcionamiento

Captura 1: Panel de monitorización de `WM_Central` mostrando las estaciones en colores Verde (Disponible), Rojo (Fuga/Avería) y Gris (Desconectada)



Captura 2: Consola de un Operario (`WM_FO`) recibiendo el resumen de un riego y alertas globales de fuga (`ALERTA#ALL`) a través de Kafka



```
--------OPCIONES--------
 1. INICIAR RIEGO
 2. PARAR RIEGO
 3. EJECUTAR SERVICIOS DEL FICHERO
 4. LISTAR ESTACIONES
 5. SALIR
¿QUÉ QUIERES HACER?
[ALERTA GLOBAL] ¡FUGA detectada en la estación WS_03!
```

Captura 3: Consola del Central mostrando la simulanción un fallo de hardware (paso a estado KO)

```
========================================
         PANEL DE CONTROL CENTRAL
========================================
1. Panel de estaciones en vivo
2. Iniciar riego       (ID de estación, o * para todas)
3. Detener riego       (ID de estación, o * para todas)
4. Bloquear estación   (ID de estación, o * para todas)
5. Activar estación    (ID de estación, o * para todas)
========================================
Selecciona una opción:
[MENSAJE de ('185.101.163.123', 53656)]: ESTADO#WS_02#FUGA
[ALERTA] la estación WS_02 ha detectado una fuga
```